

Patrón: Sección Crítica

Enfoque II — Inicialización Temprana (Early Initialization)

La **inicialización temprana** es una alternativa al uso de `synchronized` en el método `getFileLogger()`. En lugar de postergar la creación de la instancia hasta la primera llamada, la variable `logger` se inicializa directamente en su declaración, antes de que cualquier hilo pueda acceder a ella.

Motivación: coste de la sincronización

Sincronizar métodos tiene un impacto considerable en el rendimiento de la aplicación: un método sincronizado puede ser hasta **100 veces más lento** que su equivalente no sincronizado. Cuando la creación del objeto no depende de ninguna condición en tiempo de ejecución, la inicialización temprana elimina por completo esa sobrecarga.

Implementación con inicialización temprana

```
public class FileLogger implements Logger {  
  
    // Inicialización temprana: la instancia se crea al cargar la clase  
    private static FileLogger logger = new FileLogger();  
  
    private FileLogger() { }  
  
    public static FileLogger getFileLogger() {  
        return logger;    // Sin verificación, sin sincronización  
    }  
  
    public synchronized void log(String msg) {  
        FileUtil futil = new FileUtil();  
        futil.writeToFile("log.txt", msg, true, true);  
    }  
}
```

¿Por qué es seguro para hilos?

La JVM garantiza que la inicialización de campos estáticos ocurre de forma atómica durante la carga de la clase. Esto significa que `logger` estará completamente inicializado antes de que cualquier hilo pueda invocar `getFileLogger()`. No existe ninguna condición de carrera porque no hay verificación ni asignación en tiempo de ejecución.

Comparación entre ambos enfoques

Enfoque I (Sección Crítica / `synchronized`):

- Inicialización diferida: la instancia se crea solo cuando se necesita.
- Mayor sobrecarga: cada llamada adquiere un bloqueo.

- Útil cuando la creación del objeto es costosa y puede no ser necesaria.

Enfoque II (Inicialización temprana):

- La instancia se crea al cargar la clase, sin importar si se usa.
- Sin sobrecarga de sincronización en tiempo de ejecución.
- Preferible cuando el objeto siempre se necesita y su construcción no tiene efectos secundarios críticos condicionados.

Cuándo elegir cada enfoque

Use la **inicialización temprana** cuando:

- El singleton siempre será utilizado durante la vida de la aplicación.
- La creación del objeto es rápida y sin efectos secundarios no deseados al arranque.

Use la **Sección Crítica** (synchronized) cuando:

- La creación del objeto es costosa o tiene efectos secundarios (p. ej., abrir un socket) y puede no ser necesaria.

Preguntas de práctica

1. Diseñe una clase de conexión a base de datos como un singleton seguro para hilos usando inicialización temprana.
2. Diseñe una clase de cola de impresión (*spooler*) como un singleton seguro para hilos usando inicialización temprana.